Projektuppgift

Introduktion till webbutveckling med HTML, CSS och JavaScript-DT224G

Kajsa Widén

MITTUNIVERSITETET
Institutionen för Informations-system och teknologi

Författare: Kajsa Widén, kawi2602@student.miun.se
Utbildningsprogram: TWEUG, Webbutveckling, 120 hp
Huvudområde: Datateknik
Termin, år: HT, 2026

1 Sammanfattning

Målet med detta projeket har varit att företaget Äppelängens bigårdar ska få en fin och tydlig hemsida som lockar till sig kunder. Detta företaget sätter ut bikupor och tar hand om bina på andra företags marker. På hemsidan kan man bland annat läsa om vad bin gör för samhället, och hur man kan ta hjälp av Äppelängens bigårdar för att hjälpa naturen och bina. Samt hur man kan komma i kontakt med företaget.
Det finns ett kontaktformulär på hemsidan, detta formulär blir validerat när besökaren försöker skicka iväg det. Om det går igenom validationen korrekt skickas denna text till företagets ägares gmail via Netlifys hemsida.
I detta projekt har jag använt mig av Figma för att måla upp den struktur och design jag vill ha på hemsidan, och har sedan använt detta som en mall när jag har skrivit koden i Visual Studio Code. Jag har även använt mig av Git för versionshanera koden och sedan lagat det i en GitHub-repsitory. Hemsidan är upplagd på Netlifys hemsida, den är kopplad till GitHub, såfort GitHub-repositoryn uppdateras så skickas det också vidare till Netlify. Då visas ändringarna på hemsidan nästa gång den laddas om.

2 Innehållsförteckning

1 Sammanfattning ii
2 Innehållsförteckning iii
3 Introduktion 1
3.1 Bakgrund och problemmotivering 1
3.2 Avgränsningar 1
3.3 Detaljerad problemformulering 1
3.4 Författarens bidrag ..... **Error! Bookmark not defined.**
4 Teori 2
5 Metod 4
6 Konstruktion 6
7 Resultat 10
8 Slutsatser 11

3 Introduktion

3.1 Bakgrund och problemmotivering

Företaget Äppelängens bigårdar behöver en hemsida för att kunna marknasföra deras produkter och tjänster. Detta företaget sätter ut bikupor och tar hand om bina på andra företags marker. På hemsidan ska man bland annat kunna läsa om varför bin är viktigt för samhället, vad Äppelängens bigårdar kan göra för företag och hur man tar kontakt med Äppelängens bigårdar. Denna hemsidan ska fungera till olika storlekar på skärmar.

3.2 Avgränsningar

Planen var att få med företagets instagramflöde på hemsidan, jag skulle koppla denna genom API och få upp deras tre nyaste inläggen på hemsidan. Då jag kännde att jag inte hann med detta så utblev det från hemsidan.

3.3 Detaljerad problemformulering

På en av sidorna ska det finnas ett kontaktformulär där besökaren får skrvia in namn, epostadress och ett meddelande. Dessa tre fälten ska sedan valideras. När alla fält är validerade korrekt så ska formuläret skickas till Äppelängens bigårdars epostadress.

Sedan ska det också finnas ett bildspel. En knapp på var sida av bilden och så ska det gå och bläddra bland bilderna. Detta kan man koda med hjälp av JavaScript. Om man är på sista bilden i bildspelet ska det börja om från början igen, det ska också gå att backa från första bilden i bildspelet till den sista bilden. Det ska också stå villket nummer det är på bilden som visas.

I footern på små skärmar ska sakerna ligga fint vertikalt under varandra, medans på stora skärmar ska de ligga horisontellt brevid varandra och då ska den översta delen ligga i mitten.

En del av hemsidan som ska ha grön bakgrundsfärg så ska det ligga två element jämte varandra, en text och en bild. Nästa rad ska det vara tvärt om, en bild och en text. Det ska vara lätt att bygga på den här delen och fortsätta med fler rader, varanan är tvärt om än den första. Men på små skärmar ska det enbart vara vertikalt, och bilden ska alltid vara över texten, aldrig tvärt om.

4 Teori

HTML är ett språk som används för att struktuera innehåll på webben. [1]

CSS är ett språk som används för att presentera webbsidor. Bestämmer hur hemsidan ska se ut. [2]

JavaScript är ett progameringsspråk, det används för att skapa funktionalitet och logik i olika typer av applikationer. [3]

Figma är en program där man kan designa och måla upp en prototyp av en hemsida. [4]

Wireframes är en enkel skiss över webbplatsens struktur och funktion. [5]

Mockup är en mycket mer detaljerad skiss än vad en wireframe är. Ger en bild om hur en färdig webbplats ser ut. [5]

Visual Studio Code ett program som man kan skriva och testa kod i. [6]

Git är ett distribuerat versionshanteringssystem, där varje utvecklare har en kopia av hela projekthistoriken lokalt på sin dator. [7]

GitHub-repository innehåller alla projektets filer samt historiken över ändringar i projektet. [7]

JPEG (JPG) är det vanligaste filformatet för fotografier. Detta formatet går att komprimera. [8]

PNG är ett pixelformat för bilder som är lämplig att använda till t.ex. loggotyper och knappar. Går också att få bilderna transparenta. [8]

Netlify en webbsida som används för att publicera och en webbplats på internet. [9]

Mobile fist är att man bygger upp hemsidan för mobilskärmar först, det är först efter detta man bygger hemsidan till större skärmar. [10]

Media queries låter oss applicera CSS om speciella vilkor har uppfyllts. [10]

Breakepoint är när en viss skärmstorlek uppnår, då ändras layouten. [10]

CSS-egenskaper används för att betsämma hur något ska se ut på hemsidan. [11]

HTML-element är den grundläggande byggstenen i en webbsida. Det består vanligtvis av en öppningstagg, ett innehåll och en stängningstag. [12]

Flexbox är ett layoutsystem som elementen antingen kan byggas på huvudaxeln, horisontellt, eller tväraxeln, vertikalt, varje gång den används. [13]

W3C är en hemsida som validerar olika sorters kod och ger eventuella felmeddelanden. [14]

Event lyssnare skrivs i JavaScript för att reagera på användarens handlingar, t.ex. trycka på knappar eller fylla i ett formulär. [15]

CSS Grid är ett tvådimensionellt layoutsystem, man kan alltså få innehållet i rutnät, horisontellt och vertikalt samtidigt. [16]

5 Metod

--Figma ska användas för att skapa wireframes för både stor och liten skärm, samt en mockup för att få fram de rätta färgerna och känslorna till hemsidan.

--All kod ska skrivas i Visual Studio Code och versionshanera i med hjälp av Git. Ett GitHub-repository kommer också användas för att lagra allt i. Den färdiga webbsidan ska läggas upp på Netlify.

--Bilderna till hemsidan ska vara JPEG-format, men loggotyperna ska vara i PNG-format. Detta är bland annat för de kan få transparent bakgrund. Till exempel så vill vi ha loggotypen nere vid footern transparent, för att det ska bli så snyggt som möjligt.

--Kontaktformuläret ska skrivas i HTML och ändra layouten i CSS. JavaScript kommer användas för att validera formuläret. Varje fält kommer valideras med en if-sats som kontrollerar om fältet är tomt. Epostfältet valideras också genom HTML för att se om mailen som anges är ett godkänt epostformat. Detta är viktigt att validera för att det ej ska skickas iväg ett halvfärdigt formulär. De sakerna som valideras behövs för att företaget sedan ska kunna återkoppla till den som skickade formuläret.

Om valideringen är godkänt på alla tre fälten ska formulärets skickas vidare till en Äppelängens bigårdars epostadress. Detta måste gå genom en server för att det ska fungera. Jag anmänder mig utav Netlifys formhantering. Netlify tar informationen från formuläret, sätter ihop det till ett mail och vidarebefodrar detta till angiven epostadress, som i detta fallet är sebastian@appelangen.se.

--Det ska finnas ett bildspel på förstasidan så att företagen kan få en liten större inblick på vad Äppelängens bigårdar håller på med. Det blir mycket tydligare och verkligare om man både får läsa och se bilder. Bildspelet ska skrivas i HTML, sedan ska CSS ändra layouten. På små skärmar ska knapparna för att byta bild vara under bilden, och ett paragrafelement mellan dessa två knapparna som visar vilket bildnummer det är som visas just nu. JavaScript kommer användas för att det ska fungera att byta bild och bildnummer med hjälp av knapparna. Det kommer finnas en array med flera objekt i med två egenskaper i varje objekt, bild och alt-text till bilen. När det trycks på en knapp kommer JavaScript ta upp en if-sats om den är på den första eller sista bilden i arrayen, beroende på vilken knapp man trycker på.
Sedan byts det till en annan bild, alt-text och bildnummer, vilken det blir är vad svaret på if-satsen är.

--Footern ska skrivas i HTML, i den ordningen som delarna ska ligga i på små skärmar, eftersom vi använder oss av mobilde first. Sedan ska det i CSS ändra layouten och med hjälp av media queries. Det ska finnas en breakepoint vid 768 pixlar då layouten i CSS istället ska anpassa sig till stora skärmar. Då ska de tre

delarna, som ligger i var sin div byta plats med hjälp av CSS-egenskapen order. Jag vill att loggan är det första man tittar på när man kommer till footern. Då känns det mest logist att loggan är det första som kommer på de mindre sidorna och att de är i mitten på de större.

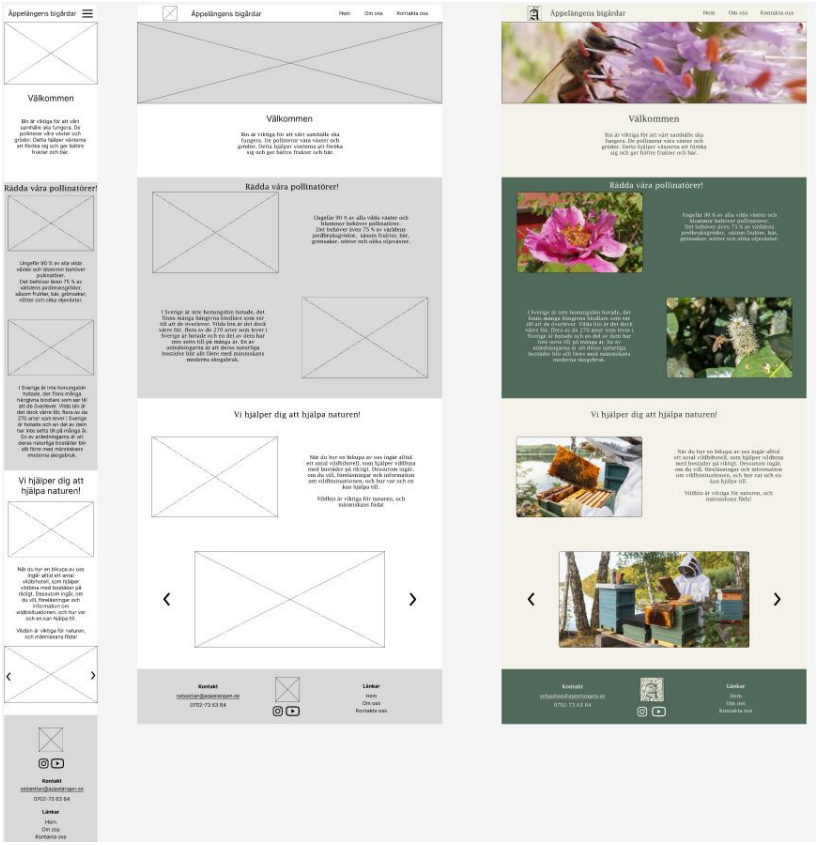
--I den delen av sidan som ska ha en grön bakgrunsfärg så ska koden skrivas i HTML för små skärmar, då vi använder oss av mobile first. I CSS ska de stylas. CSS ska också hjälp med media queries göra en breakepoint vid 768 pixlar för större skärmar. Här ska ett HTML-elementen delas in i flexboxs två och två, en bild och en text. Varanan flexbox ska vara tvärt om från den första, de ska ha värdet row-reverse för att detta ska fungera. Detta vill jag ha för att det ska vara tydligt och lätt att veta vart man ska titta någonstans. Men det ska inte heller bli för enformigt så man tröttnar.

--HTML- och CSS-koden ska valideras utan fel i W3C valideringssystem för att koden ska bli så korrekt som möjligt.

--På den färdiga hemsidan ska jag också kolla så alla länkar till respektive HTML-sidor fungerar, jag ska också testa så länkarna till andra sidor också fungerar och att det öppnas i en ny flik. Bildspelets knappar ska också testast så det blir ny bild, och nytt bildnummer. Detta bildspel ska fortsätta visa bilder, oavsett om man är på den första eller sista bilden. Det ska gå som en loop.

6 Konstruktion

Jag har skapat en design i Figma som visas i figur 1. Denna är den första bilden jag hade över hemsidan och jag har följt denna som en mall för hur jag vill att hemsidan ska se ut. Jag har dock gjort lite andra val på vissa saker, som t.ex. bildspelet. Här har jag lagt in en text om vilken bild man är på, och av hur många bilder som finns. Tänker det kan vara trevligt av besökaren att veta detta. Sedan har jag också valt att sätta sidoknapparna undertill istället för på bilden på de midre skärmarna. Detta är för att jag vill att så mycket bild som möjligt ska visas då bilden redan är såpass liten. Jag har också ändrat loggan längst upp till ett annat typ av Ä, för jag kände att det blev lite för klottrigt med det första alternativet när den var så liten. Sedan har jag också gjort så att denna loggan är en del av företagets namn.



Figur 1, wireframes och mockup.

--Formuläret har validerats med JavaScript med if-satser, först är det en if-sats runt event lyssnaren, som gör att det bara är den HTML-sidan som formuläret är på lyssnar på skicka knappen. När det trycks på skickaknappen så går JavaScript igenom fält per fält med en if-sats vid varje fält, som ni ser i figur 2. Här ser den till att inga fält är tomma, eller att det bara står mellanslag i. När detta är klart används Netlify formhantering för att sedan viderbefodra mailet till sebastian@appelangen.se.

```
//Läser om de har skrivit in något namn
if (name === ""){
    errors.push("Du måste ange ditt namn");
    validate = false;
}
//Läser om de har skrivit in någon e-postadress
if (mail === ""){
    errors.push("Du måste ange din e-postadress");
    validate = false;
}
//Läser om de har skrivit något meddelande
if (message === ""){
    errors.push("Du måste skriva ett meddelande");
    validate = false;
}
```

Figur 2, koden som validerar kontaktformuläret.

--Bilspelet visar en bild, och det står vilken bild man är på under bilden. Bilden och räknaren undertill byts när man trycker på någon av sidoknapparna, om man trycker på höger blir det nästa bild, och om man trycker på vänster blir det föregående bild. Tanken först var att när man trycker på en knapp så skulle JavaScript ta bort hela bildelementet, och sedan lägga till en ny bild. Här var inte bildräknaren heller med i tankarna.

Men sedan kom jag på att jag kunde ha en array med tre objekt i, bilden, alt-texten och bildräknaren. Detta sättet kunde skrivas mycket tydligare så det är mycket lättare att läsa denna delen av koden, därför valde jag att göra såhär istället. Jag började med denna kod, men upptäckte snabbt att det skulle bli väldigt jobbigt om man lägger till eller tar bort någon bild för då behöver man ändra varje bildräknare i arrayen. Jag löste detta genom att JavaScript fick räkna ut vilket nummer det var på bilden, och hur många bilder det finns totalt och skriva ut detta, figur 3 visar denna koden.

```
/* Räknar och skriver ut vilken bild det är som visas,
och hur många som finns totalt */
slideCounter.innerHTML = `${nowPic+1}/${pictures.length}`;
```

Figur 3, kod som räknar och skriver ut bildnummret.

När man trycker på högerknappen, så ska man komma till nästa bild. Då är det en if-sats som räknar ut om man är på sista bilden. Om svaret är sant byts index ut till 0, alltså första bilden. Om svaret är falskt ska index bara plussas på med 1 istället, så nästa bild visas.

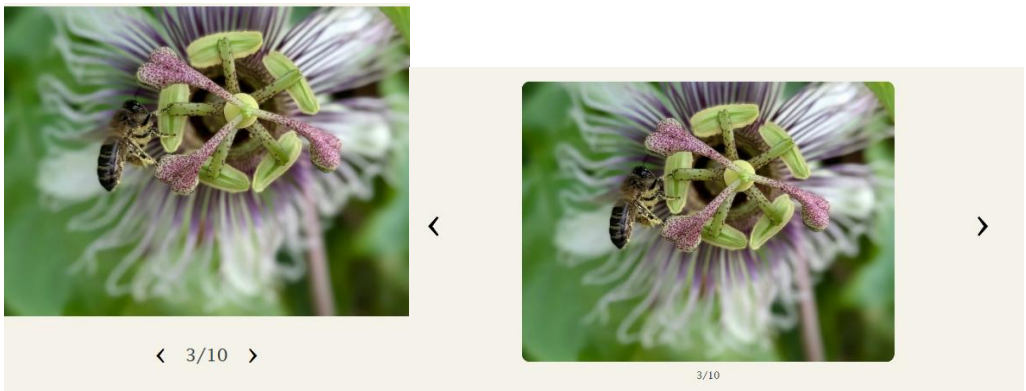
Om man istället trycker på vänsterknappen för att komma till förgående bild så

är det en annan if-sats som räknar ut om det är den första bilden som visas. Om svaret är sant så visas den sista bilden som finns i arrayen upp. Men om svaret är falskt så visas förgående bild med index minus 1. Figur 4 visar hur korden ser ut när man trycker på vänsterknappen. Jag vill inte att bildspelet skulle ta slut åt något håll utan fortsätta loopa för det känns större då. Det blir inte heller lika jobbigt om man är på sista bilden och vill bläddra till bild nummer två.

```
if (nowPic === 0 ) {
    // Om arrayen är på första bilden, byt till den sista bilden i arrayen
    nowPic = pictures.length -1;
} else {
    // Om ej, byt till den förra bilden
    nowPic--;
}
```

Figur 4, kod som räknar ut om det är bild nummer ett.

På små skärmar visas dessa knapparna och bildnumret under själva bilden, fast på större skärmar så är knapparna på var sin sida om bilden, fast bildnumret stannar kvar under bilden, figur 5 visar bilspelet i de två olika storlekarna. Detta löstes med hjälp av CSS Grid. Då placerades knapparna, bilden och bildnumret på de platserna de ska ha. Det är också kodat så att sidoknapparna anpassar sig till hur stor bilden är. Jag använde CSS Grid här för jag ville ha två rader med tre kolumner. Då kändes det givet att ta CSS Grid.



Figur 5, bildspelet på små respektive stora skärmar.

--Footern är uppdelad i tre olika divs i HTML, första med logga och sociala medier, andra med kontaktuppgifter och den tredje med länkar till de andra sidorna på hemsidan. På små skärmar är alla tre i en flexbox som har riktningen vetikalt och de är centerade i mitten av skärmen. Däremot på stora skärmar så är denna flexbox istället horisontell, och nu vill vi få den div med loggan i mitten istället för först. Då använder vi värdet *order* för att byta plats på diven, demostration vid figur 6. Detta kändes som den minst komplicerade sättet att göra detta på. Det är väldigt rent och snyggt vid CSS-kodningen vid denna delen så det är lätt att följa med på vad som händer.

```css
.logo-media {
    order: 2;
}

.contact {
    order: 1;
}

.links {
    order: 3;
}
```

Figur 6, hur elementen byter plats.

--Den delen med grön bakgrund vid hemsidan har jag använt HTML för att bygga koden och gruppera de delarna jag vill ska ligga jämte varandra på större skärmar i olika divs. Varje div består av en bild och en text. När skärmen blir större än 768 pixlar så läggs var div horizontellt med hjälp av flexbox. Eftersom bilden alltid ligger före texten och vi vill ha det tvärt om i varannan rad så har vi en CSS-regel för just detta. Denna regel heter *reverse* och är också en flexbox. Men värdet till denna flexbox är istället *row-reverse*. Så varje element som man ger klassen *reverse* till får denna egenskapen. Då blir det lättare att välja vilka man vill ska vara tvärt om mot de andra. Det kändes bra att använda flexbox här för att då är det lätt att välja vilka man vill ha åt det vanliga hållet eller det omvända.

7 Resultat

I figur 7 visas en tabell med det som behövs kontrolleras med den färdiga produkten, där står det också om kontrollen är lyckad – Ja eller misslyckad – Nej.

Kontrollera	**Ja/Nej**
Skapat wireframes och mockup till hemsidan	Ja
Hemsidan är anpassad till olika skärmstorlekar	Ja
Navigation inom hemsidan fungerar	Ja
Navigation ut från hemsidan fungerar	Ja
Formulärvalideringen fungerar	Ja
Formuläret skickas via Netlify till det angiva epostadressen	Ja
Git användes regelbundet för att versionshantera ändringar	Ja
Publiceringen är automatiserad	Ja
HTML och CSS validerars utan fel	Ja

Figur 7, tabell som kontrollerar den färdiga produkten

8 Slutsatser

Idéen med denna produkten var att företaget Äppelängens bigårdar skulle få en egen hemsida så de kan visa upp sina tjänster och produker. Det tycker jag är lyckat! På hemsidan kan man läsa om vad bina gör för samhället, vad företaget säljer och man kan enkelt få kontakt med företaget. Det är lätt att navigera sig och hitta vad man behöver. Hela kontrollen under rubriken resultat blev godkänt så allt fungerar också som det ska.

Valideringen i HTML och CSS koden går igenom utan några problem. Detta gör det också lättare att förstå koden mer då man vet att inget där är fel. Koden är också välkommenterad så det är lättare att hitta det man söker.

En sak som känns väldigt bra att ha ändrat från originalidén är hur knapparna i bildspelet ligger på de mindre skärmarna. Att de ligger nedanför bilden istället för på själva bilden. Jag tycker att detta gör hemsidan mycket renare och tydligare.

Kodmässigt vid bildspelet blev det också mycket lättare när varje bild har ett objekt med bildlänk och alt-texten. Det blir mer struktuerad och lättare att läsa. Sedan att bildnummret räknas ut av JavaScript underlättar det också väldigt mycket så det inte råkar bli fel någonstans när man tar bort någon bild eller liknande. Denna lösningen är jag väldigt nöjd över.

Jag tycker också att footern blev snygg. Det är lagom mycket information i den, så man enkelt kan se vad som finns, och man kan lätt hitta kontaktuppgifterna till företaget. Kontaktformuläret är också väldigt tydlig och den klarade valideringstestet, det kan man se under rubriken Resultat.

Den gröna sektionen gör det som jag vill, att varannan rad ska vara tvärt om från den första. Men där blev jag ändå missnöjd, för jag lyckades inte få till hela den layouten som jag egentligen ville ha. Jag vill att bilderna och texterna ska ligga mer som kanterna i en fyrkant, att de är lite symmetriskt, men jag lyckades ej med detta. Tänker att CSS Grid kanske hade varit bättre att använda här.

Som helhet är jag nöjd över hemsidan, det är färger som går åt bin och naturhållet, och det är lätt att hitta det man letar efter. Jag har gjort ett bildspel som blev bättre än vad jag först planerade och det ser också fint ut i JavaScript-koden, där det är struktuerat.

Källförteckning

[1] Canvas, ”Introduktion till HTML”
https://canvas.miun.se/courses/11718/pages/introduktion-till-html?module_item_id=230249
Publicerad 2026-08-31. Hämtad 2026-10-31.

[2] Canvas, ” Introduktion till CSS”
https://canvas.miun.se/courses/11718/pages/introduktion-till-css?module_item_id=231178
Publicerad 2026-08-31. Hämtad 2026-10-31.

[3] Canvas, ”Introduktion till JavaScript”
https://canvas.miun.se/courses/11718/pages/introduktion-till-javascript?module_item_id=230266
Publicerad 2026-08-31. Hämtad 2026-10-31.

[4] Figma, ”What is Figma?”
https://help.figma.com/hc/en-us/articles/14563969806359-What-is-Figma
Hämtad 2026-10-31.

[5] Canvas, ”Designprocess – wireframes, mockups och prototyper”
https://canvas.miun.se/courses/11718/pages/designprocess-wireframes-mockups-och-prototyper?module_item_id=231192
Publicerad 2026-08-31. Hämtad 2026-10-31.

[6] Visual Studio, ”Visual Studio Code”
https://code.visualstudio.com/?_exp_home=e855cfac50
Hämtad 2026-10-31

[7] Canvas, ”Introduktion till Git”
https://canvas.miun.se/courses/11718/pages/introduktion-till-git?module_item_id=230260
Publicerad 2026-08-31. Hämtad 2026-10-31.

[8] Canvas, ”Bilder och media”
https://canvas.miun.se/courses/11718/pages/bilder-och-media?module_item_id=230253
Publicerad 2026-08-31. Hämtad 2026-10-31.

[9] Netlify, ”About Netlify”
https://www.netlify.com/about/
Hämtad 2026-10-31.

[10] Canvas, ”Media queries och mobile first”
https://canvas.miun.se/courses/11718/pages/media-queries-och-mobile-first?module_item_id=231189

Publicerad 2026-08-31. Hämtad 2026-10-31.

[11] W3Schools, ” CSS @property-regel”
https://www.w3schools.com/css/css3_property.asp
Hämtad 2026-10-31.

[12] Canvas, ”HTML-element”
https://canvas.miun.se/courses/11718/pages/html-element?module_item_id=230251
Publicerad 2026-08-31. Hämtad 2026-10-31.

[13] Canvas, ”Flexbox”
https://canvas.miun.se/courses/11718/pages/flexbox?module_item_id=231187
Publicerad 2026-08-31. Hämtad 2026-10-31.

[14] W3C, ”Markup Validation Service”
https://validator.w3.org/#validate_by_uri
Hämtad 2026-10-31.

[15] Canvas, ”Events (händelsehantering)”
https://canvas.miun.se/courses/11718/pages/events-handelsehantering-2?module_item_id=256980
Publicerad 2026-08-31. Hämtad 2026-10-31.

[16] Canvas, ”Css Grid”
https://canvas.miun.se/courses/11718/pages/css-grid?module_item_id=231188
Publicerad 2026-08-31. Hämtad 2026-10-31.